

Aufgabe 8

Aufgabe 8.1

Erklärung:

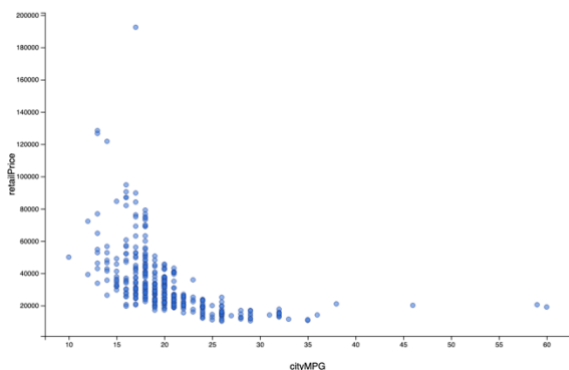
Die Idee ist, den Scatterplot aus den vorherigen Übungen so umzubauen, dass die beiden Achsen-Attribute nicht mehr im Code festverdrahtet sind, sondern zur Laufzeit per Dropdown aus den sieben geforderten Größen (cityMPG, retailPrice, dealerCost, carLen, weight, carWidth, engineSize) gewählt werden können — ohne die Plot-Logik anzufassen. Dafür wird zuerst ein schlanker FilteredCar-Record gebaut, der genau diese sieben Attribute als Float enthält. toFilteredCar füllt ihn über eine Applicative-Pipeline mit einem selbst geschriebenen andMap : Maybe a -> Maybe (a -> b) -> Maybe b (nötig, weil Maybe.mapN nur bis N = 5 geht, hier aber 7 Felder zu kombinieren sind); sobald ein Feld Nothing ist, fällt das Auto automatisch durch List.filterMap raus — Filtern und Typ-Vereinheitlichen passieren in einem Schritt. Die wählbaren Achsen liegen in attributes : List Attribute, wobei Attribute Name und Zugriffsfunktion bündelt (Attribute "retailPrice" .retailPrice); ein Helper findAttribute übersetzt den <select>-Wert in das passende Attribute zurück. Model = { xAttr, yAttr } und Msg = SelectX String | SelectY String reichen, weil filteredCars als Top-Level-Konstante einmalig berechnet wird. Der eigentliche scatterplot weiß nichts von Dropdowns — er holt sich aus dem Modell zwei Attribute-Werte und benutzt deren get-Funktion sowohl für Scale.linear (Wertebereich über niceExtent mit 5 % Padding) als auch für cx/cy der Punkte und für das Achsen-Label, sodass Skala, Tick-Beschriftung, Punktposition und Label garantiert synchron bleiben. Eine achte Größe ergänzen heißt damit nur: ein Eintrag in attributes, ein Feld im FilteredCar, ein zusätzliches andMap — die Plot-Logik bleibt unangetastet.

Screenshots:

Aufgabe 8.1 – Scatterplot

Es werden 387 Autos angezeigt (alle Autos ohne Wert in einem der sieben Attribute wurden entfernt).

X-Achse: Y-Achse:



Link: <https://ellie-app.com/yN37sq6qVyya1>

Aufgabe 8.2

Erklärung:

Die Idee ist, alle 11 numerischen Attribute eines Autos auf die Winkel der Limbs einer Stick Figure nach Pickett & Grinstein (1988) abzubilden und diese Figuren als Punkte in den Scatterplot aus 8.1 zu setzen. Die Figur besteht aus einem vertikalen Rumpf und drei „Hubs“ (oben / Mitte / unten), an denen insgesamt 11 Limbs hängen — fünf oben (Kopf + vier Arme), zwei in der Mitte und vier unten (Beine). Jeder Limb wird durch eine `LimbSpec { label, hub, baseAngleDeg, rangeDeg, getter }` beschrieben: der Basis-Winkel legt die „Grundpose“ der Figur fest, der Bereich begrenzt den erlaubten Schwenk; der konkrete Winkel ist $\text{base} + (n - 0.5) \cdot 2 \cdot \text{range}$ mit $n \in [0, 1]$ als lineare Min-/Max-Normalisierung des Attributs über alle gefilterten Autos. Endpunkte werden direkt mit $\cos/\sin$ berechnet ($x = o_x + s \cdot \cos \alpha$, $y = o_y - s \cdot \sin \alpha$, das Minus wegen SVG-y-nach-unten), als `TypedSvg-line-Segmente` gezeichnet und in einer einzigen Funktion `stickFigureLines cx cy s sw color car` zusammengefasst — die wird sowohl von der großen Legenden-Figur (für das erste Auto, mit nebenstehender Tabelle, die Wert, Hub, Basis-Winkel, Winkel-Bereich und aktuellen Winkel pro Attribut nennt) als auch vom Scatterplot benutzt, der pro Auto eine Mini-Figur an (`Scale.convert xScale`, `Scale.convert yScale`) zeichnet. Das `carType` der Autos wird zusätzlich auf die Farbe der Figur gemappt (6 Klassen, separate Farb-Legende). Damit das in der Aufgabenstellung verlangte Texturexperiment funktioniert, hat das Modell ein drittes Feld `stickSize : Float`, das per Range-Slider live geändert wird; je nach Größe sieht man entweder einzelne Figuren oder kollektive Muster, die sich in dichten Punktwolken bilden — der Plot selbst muss dafür nichts wissen, er bekommt die Größe einfach als Parameter durchgereicht.

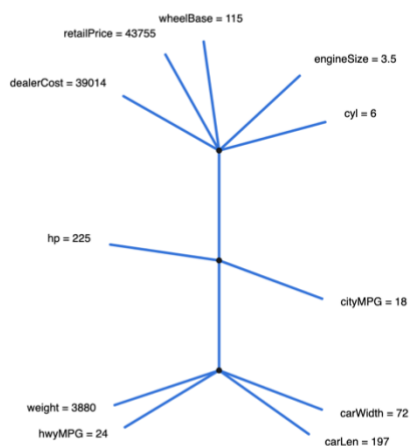
Screenshots:

Aufgabe 8.2 — Stick Figure (Pickett & Grinstein 1988)

Es werden 387 Autos angezeigt (alle Autos ohne Wert in einem der 11 numerischen Attribute wurden entfernt).

Legenden-Figur — Acura 3.5 RL 4dr

Typ: Sedan/Kompakt — Farbe der Figur.

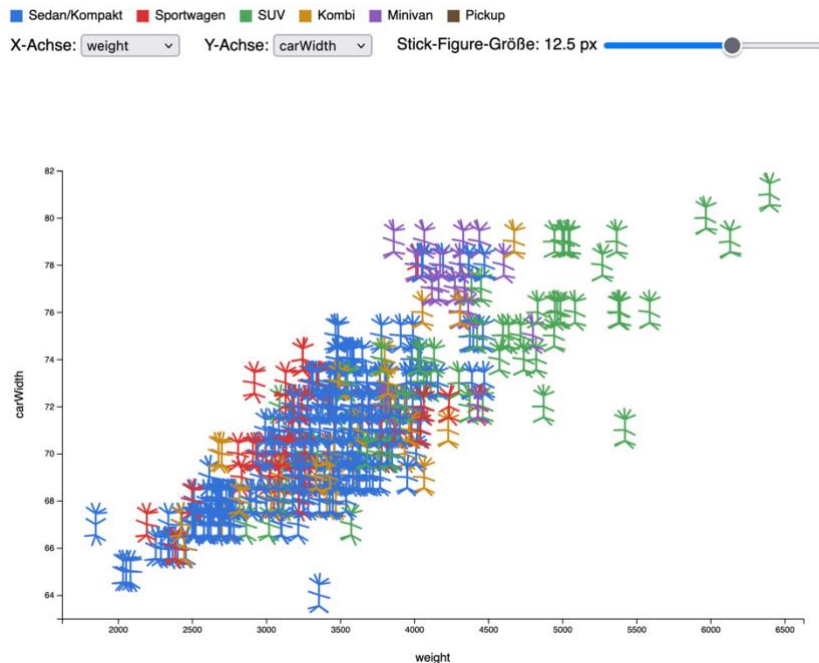


Attribut → Winkel-Mapping

Jedes Attribut wird linear auf $[0, 1]$ normiert (über alle 387 gefilterten Autos) und dann auf den angegebenen Winkel-Bereich um den Basis-Winkel abgebildet.

Attribut	Wert	Hub	Basis-Winkel	Winkel-Bereich	akt. Winkel
wheelBase	115	oben	90°	±30°	98.05°
retailPrice	43755	oben	135°	±25°	119.19°
dealerCost	39014	oben	160°	±15°	150.34°
engineSize	3.5	oben	45°	±25°	42.83°
cyl	6	oben	20°	±15°	15°
hp	225	Mitte	180°	±30°	171.71°
cityMPG	18	Mitte	0°	±30°	-20.4°
hwyMPG	24	unten	225°	±25°	211.11°
weight	3880	unten	200°	±15°	198.38°
carLen	197	unten	315°	±25°	324.62°
carWidth	72	unten	340°	±15°	339.12°

Scatterplot mit Stick Figures statt Kreisen



Link: <https://ellie-app.com/yN3jFmVs4TSa1>

Aufgabe 8.3

Erklärung:

Die Idee ist, jedes Auto als komplettes Icon im Scatterplot aus 8.1 darzustellen: 11 numerische Attribute werden zu den 11 Achsen eines Sterns, der Wert auf jeder Achse zu einem Polygon-Eckpunkt, und die Eckpunkte zu einem geschlossenen Polygon verbunden. Die Achsen liegen in `axisSpecs : List (String, FilteredCar -> Float)`; die i -te Achse hat den Math-Winkel $\text{axisAngle } i = \pi/2 - 2\pi \cdot i / 11$, sodass Achse 0 nach oben zeigt und der Rest im Uhrzeigersinn im Abstand von $\approx 32,7^\circ$ folgt — das wird einmal vergeben, nicht pro Auto. Damit die Polygone vergleichbar sind, wird jeder Achswert über `normalize linear` auf $[0,1]$ über alle gefilterten Autos abgebildet (gleicher Filterschritt wie in 8.2: `andMap`-Pipeline + `List.filterMap`, fertige `FilteredCar`-Werte als Top-Level-Konstante). Die Funktion `starPoints cx cy s car` baut daraus per `List.indexedMap` die Liste der Eckpunkte $(c_x + s \cdot n \cdot \cos \alpha, c_y - s \cdot n \cdot \sin \alpha)$ — Math-Konvention, mit Minus bei $\sin$ für SVG-y-nach-unten — und übergibt diese direkt einem einzigen `TypedSvg.polygon` mit halbtransparentem Fill (28 % Alpha) und farbigem Rand nach `carType`. Genau dieselbe `starIcon`-Funktion wird zweimal benutzt: einmal groß ($s = 150 \text{ px}$) für die Legenden-Figur des ersten Autos — zusammen mit konzentrischen Hilfskreisen bei 0,25/0,5/0,75/1,0, allen Achsenlinien, Achsenbeschriftungen am äußeren Ende und einer Tabelle (Wert, Winkel, normierter Wert) —

und einmal klein im Scatterplot, wo pro Auto ein Icon an (Scale.convert xScale, Scale.convert yScale) gezeichnet wird. Wie in 8.2 hat das Modell ein drittes Feld iconSize : Float, das per Range-Slider (3–30 px) live geändert wird, damit man wie verlangt mit der Größe experimentieren kann — kleine Icons zeigen wieder kollektive Muster, größere die individuelle Form jedes Vektors.

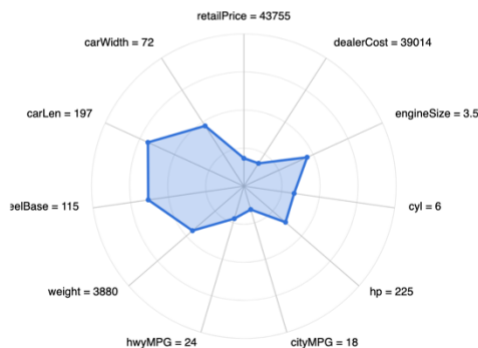
Screenshots:

Aufgabe 8.3 — Star Plot Icons

Es werden 387 Autos angezeigt (alle Autos ohne Wert in einem der 11 numerischen Attribute wurden entfernt).

Legenden-Star-Plot — Acura 3.5 RL 4dr

Typ: Sedan/Kompakt. Die 11 Achsen sind gleichmäßig auf 360° verteilt ($\Delta \approx 32.73^\circ$). Jeder Achswert ist linear auf [0,1] normiert; konzentrische Hilfskreise bei 0,25 / 0,5 / 0,75 / 1,0.



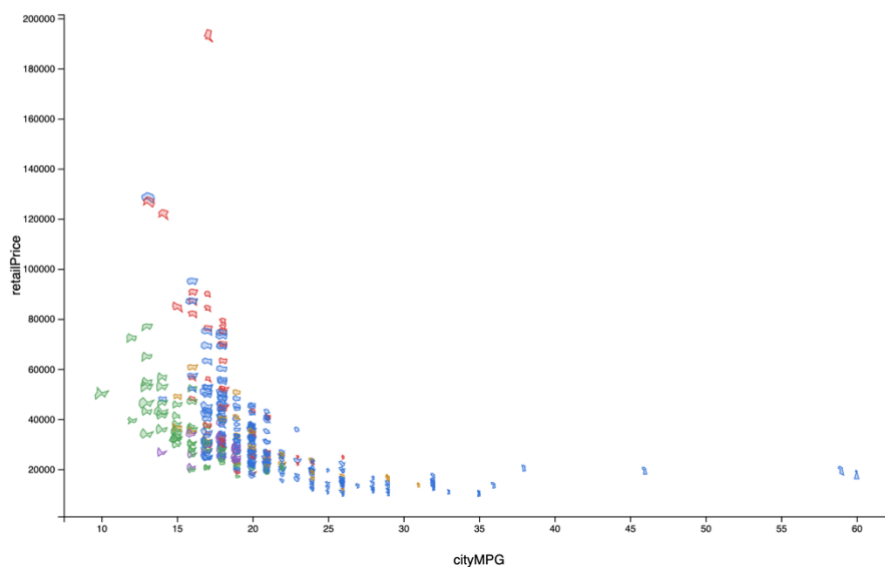
Achse → Wert

Achse	Attribut	Wert	Winkel	norm.
1	retailPrice	43755	90°	0.18
2	dealerCost	39014	57.27°	0.18
3	engineSize	3.5	24.55°	0.46
4	cyl	6	-8.18°	0.33
5	hp	225	-40.91°	0.36
6	cityMPG	18	-73.64°	0.16
7	hwyMPG	24	-106.36°	0.22
8	weight	3880	-139.09°	0.45
9	wheelBase	115	-171.82°	0.63
10	carLen	197	-204.55°	0.69
11	carWidth	72	-237.27°	0.47

Scatterplot mit Star-Plot-Icons statt Kreisen

■ Sedan/Kompakt ■ Sportwagen ■ SUV ■ Kombi ■ Minivan ■ Pickup

X-Achse: cityMPG Y-Achse: retailPrice Icon-Größe: 8 px



Link: <https://ellie-app.com/yN3vT536P3ra1>